電腦對局理論 Final

蔡平樂

January 3, 2026

1 Implementation detail

1.1 Main Searcg Algorithm

---

procedure NEGAMAX(pos, depth)
    is_stable ← is_quiescence_state(pos)
    limit_depth = $\begin{cases} 0 & if \text{ is\_stable} \\ \text{quiescence\_limit} & else \end{cases}$
    if depth ≤ limit_depth **then**
        return Evaluate(pos)
    moves ← gen_move(pos)
    moves ← ordering_moves(moves, depth, only_critical = is_stable)
    if moves.size() == 0 **then**
        pos.pass_move()
        return Negamax(pos, depth-1)
    opt_moves = None, opt_score = -∞
    /*performing standard Negascout w.r.t. moves */
    return opt_score

The function ordering_move will ordering the moves based on the move evaluate function, and filter moves base on the states of positions and depth. If *depth* <= 2, then the it will delete all Flipping moves; if **only_critical** is true, then it will remove all non-capture and non-escape moves, only leaving critical moves for quiescence search.

The **quiescence_limit** is setting to -6, allowing the algorithm performing quiescence search even if the depth is less than 0, and the maximum depth is restricted to 6. Based on experiments, if we don't restrict the depth of quiescence search(I guess may some cycle occured when searching), then the depth can be really deep if there's some Hidden pieces at the board, causing huge redundancy of searching tree.

1.2 Board Evaluation Function

I decomposite the board evaluation function into three parts: summation of piecevalues, positions structure score, and Soldier (General) structure, the Evaluation scores will be their weighted sum.

1.2.1 Pieces Value

piece values score is simply defined the differece of summation of all face-up and hidden pieces, the piecevalues is static defined as follow:

將	士	象	車	馬	包	卒
20	25	18	5	3	18	1

Pieces Value

Rather than the table in HW2, this table extend the gap of piecevalues between important pieces (將、士、象、砲) and others (馬、車、卒)

Other evaluations share the same PieceValue table, at the following report, I will use $PV[piecetype]$ to represent the PieceValue of the corresponding PieceType.

1.2.2 Distance Score

In convinience, assuming we are Black side, the value is higher if there's more advantage we have.

For a fixed piecetype R, we want to use a function $F_B(pos, R)$ to evaluate the level of ease to capture R based on the deployment of pos.

for a single black piece B and a red piece R of type T, let $d_{B,R}$ represent the Manhattan distance of $B.location$ and $R.location$, then I use $D(R, B) = \begin{cases} d_{B,R} & if\ d_{B,R} \leq 3 \\ 2 \times d_{B,R} & else \end{cases}$ to evaluate the ease for B to capture R.

Subsequently, F is constructed by $D(B, R)$:

$$F_B(pos, R) = \left[\sum_{B \in P^b_>(T)} D(B, R) + \frac{1}{2} \sum_{B \in P^b_=(T)} D(B, R) \right]$$

Where

- $P^b_>(pos, T) = \{B : B\ can\ capture\ T, B.type \neq Cannon\}$
- $P_=(pos, T) = \{B : B.type = T, B.type \neq Canon\}$

The constant 20 can assure $D \geq 0$ for any combination of B, R because the diameter of board is 10, which ensure F is higher for the board of more alive black pieces.

We define $SideDisScore_B$ to estimate the ease for Black side to capture all Red pieces:

$$SideDisScore_B(pos) = \sum_T \frac{PV(T)}{2^{penalty(pos,T)}} \times \frac{\sum_{R,R.type=T} F_B(pos,R) + died(pos,Red,R) \times bound(T)}{initial(T) \times bound(T)}$$

The meaning of each notations refer to Appendix

Symmetrically, we can define $SideDisScore_R$ with the same mannar. Generally, for each side=Black or Red, the total score $dis\_score(pos, side)$ is calculated by $SideDisScore_{side}(pos) - SideDisScore_{opponent}(pos)$

1.2.3 Soldier/General score

To evaluate the score of Soldiers and General, let $pawn^b_{safe}$ = number of 卒 is hidden or is not being attacked, and $pawn^b_{danger}$ is the 卒 s which being attacked, we said a piece is attacked if there's any adjacent non-Cannon piece which can capture it.

let $pawn^b_{available} = pawn^b_{safe} + \lfloor \frac{pawn^b_{danger}}{2} \rfloor$, which is the estimation of the number of available 卒, which is usually the upperbound because half of threated 卒 can escape.

The Soldier_score of Black side is given by

$$\begin{cases} pawn^b_{available} & if \text{ 帥 } is\ captured \\ PawnScore(pawn^b_{available}) & else \end{cases} \tag{1}$$

where the value of PawnScore following the table:

number of pawns	0	1	2	3	4	5
value	-1000	-500	-10	-2	-1	0

When the number of 卒 is larger than 3, the variant of Soldier_score is not huge, so the solver can sacrifice some 卒 s to obtain other advantages. However, when there's only 1 or 2 卒 left, solver will trying to protect them because the huge drop of Soldier_score, which can help us to preserve the power for fighting with opponent 帥.

1.2.4 Evaluation Function

The evaluation function is the linear combination of the evaluations above:

$$Evaluation(pos) = w_1 \times piece\_val(pos) + w_2 \times dis\_score(pos) + w_3 \times Soldier\_score(pos)$$
$\mathrm{w}_1 = 10, \mathrm{w}_2 = \mathrm{w}_3 = 1$

1.3 Move Ordering

First, orderer fliter all illegal moves of the MoveList, the flipping-move control and quiescence search control all depend on move-ordering. Second, orderer use EvalMove function to evaluate each move, and insertion-sort them according to the evaluation value, return the sorted moves. The precise algorithm steps refer to the Appendix

For a move $move = (piece, sq_{from}, sq_{to})$, The Evaluation Score of a move is defined as follw:

$$MoveEval(move) = \Delta PieceValue(move) + \Delta SqEval(move) + history(move)$$

where

- $\Delta PieceValue(move)$ is calculated by $\begin{cases} PV[\text{piece at } sq_{to}] & if\ exist \\ 0 & if\ not \end{cases}$
- $\Delta SqEval(move) = SqEval(piece, to) - SqEval(piece, from)$, the detail show as follow
- $history(move)$ is the history-heuristic score

The detail of each term refer to Appendix.

1.4 Time Control

First, I use a integer M estimating the remain moves to the end at n^{th} steps

$$M_0 = 100$$
$$M_n = \begin{cases} M_{n-1} + 10 & if\ M_{n-1} \leq 15 \\ M_{n-1} - 1 & else \end{cases}$$

The constant 100 is based on experiment on Medium solver.
If the remain time for round n is T_n, then the thinking time will be

$$\text{thinking\_time}_n = \frac{T_n}{M_n}$$

Besides, when the solver have already search 6 layers, the it will early-stop the search if the spending time exceed the minimal thinking time $2s$.

2 Location of functions

- Negamax @ AB_agent.cpp: L219
- Star2_Evaluate @ AB_agent.cpp : L565
- ordering_move @ AB_agent.cpp: L16
- evaluate_move @ AB_agent.cpp: L134
- calculate_score(board evaluation function) @ evaluator.cpp: L3

3 Experiments

At the following experiments, the time constrints of are 30 seconds for each solver.

3.1 Effect of Negascout

Following is the result of 100 matchs between Negamax and Negascout solvers:

solver	Negamax	NegaScout
score	65	35

I'm not sure if there's bug at the code, but the performance of Negamax is better than Negascout. Therefore, the final tournament and the following experiments are based on pure negamax algorithm.

3.2 Effect of Quiescence searching

My solver for the final tornament restricting the depth of quiescence search above 6 layers.

Following is the tests for different layers configurations.

3.2.1 Experiment of with/without Quiescence Search

I use my tornament model compete with the model without permitting quiescence search, which setted by letting the limit depth as 0.

solver	depth = 6	No quiescence search(depth = 0)
score	25.5	14.5

As we can see, the win rate of 6 layer quiescence search is higher.

3.2.2 Experiments of different quiescence-search depth

A \ B	depth = 2	depth = 4	depth = 8
depth = 6	51.5 / 48.5	48 / 52	51 / 49
depth = 0	37.5 / 62.5	41 / 59	40 / 60

(score of A / score of B)

score of different quiescence depth settings

3.3 Effect of Move Ordering

Following experiments evaluate the result of ordering moves by comparing the different solvers as below (configurations except move orvering are follow the standard configuration mentioned at **Implementation Detail**):

- **standard**: The move-orderings performed as above
- **default-ordering**: Directly use the default order genrated by wakasagi engine(i.e. ordered by 將、士、象...).
- **random ordering**: Sort the moves with a randomly generated score of each move(control group)

Following is the performance of each group

A \ B	standard	default	random
standard	N/A	15.5 / 4.5	14.5 / 5.5
default	4.5 / 15.5	N/A	4.5 / 15.5
random	5.5 / 14.5	15.5 / 4.5	N/A

4 Appendix

4.0.1 Notations of SideDisScore

$$SideDisScore_B(pos) = \sum_T \frac{PV(T)}{2^{penalty(pos,T)}} \times \frac{\sum_{R,R.type=T} F_B(pos,R) + died(pos,Red,R) \times bound(T)}{initial(T) \times bound(T)}$$

- $initial(Red, T)$ is the number red pieces of type T at the beginning of the game, i.e. $initial(仕) = 2, initial(兵) = 5$
- $died(Red, T)$ is the number of died red pieces of type T
- $bound(T)$ is the rough estimation of the upperbound of $F(pos, T)$, as the following table

帥	仕	相	俥	傌	炮	兵
20×5+10	20+10× 2	20×3+10× 2	20×5+10× 2	20×7+10× 2	20×9	20×9+10×5

- $penalty_B(pos, T) = 0$ if at least 2 black pieces is strictly larger than T
- $penalty_B(pos, T) = 1$ if there's only one black piece strongly larger than T, and one piece equal to T
- $penalty_B(pos, T) = 2$ if there's only two equal black pieces and no stronger piece, or there's only one stronger piece and no equal piece.
- $penalty_B(pos, T) = 3$ if there's no stronger black piece, and strictly less than 2 equal piece.

$2^{-penalty_B}$ will decrease the value for T which is hard to capture for Black at current position, make it more focus on capturable pieces.

4.0.2 Algorithm of MoveOrdering

4.0.3 Detail of Square Evaluation

$SqEval(piece, sq)$ function is aimed to evaluate the advantage of *piece* placing at *sq*. More precisely, it evaluate the advantage by the mobility, information of adjancent/diagonal pieces of *sq*:

procedure MoveOrdering(MovesList, depth, only_critical)
  if depth <= 2 **then**
    delete all Flipping moves from MovesList
  if only_critical is True **then**
    delete all non-capture and non-escape move from MoveList
  MoveList ← Sort(MoveList, key(move) = EvalMove(move))
  return MoveList

$$SqEval(piece, sq) = Atk_{adj}(piece, sq) + 2 \times Control(piece, sq) + Mobility(piece, sq)$$

$$Atk_{adj}(piece, sq) = \sum_{P_{adj} \in adjacent\_pieces(sq)} can\_capture(piece, P_{adj}) \times PV[P_{adj}]$$

$$Control(piece, sq) = \sum_{P_{diag} \in diagonal\_pieces(sq)} can\_capture(piece, P_{diag}) \times PV[P_{diag}]$$

$$Mobility(piece, sq) = \sum_{adj \in adjacent(sq)} can\_goto(piece, adj)$$

The boolean function $can\_goto(P, sq)$ return true if the piece P can move to sq, i.e. there's empty or piece can capture the piece there.
$can\_capture(P_1, P_2)$ return true if P_1, P_2 not at the same side, and P_1 can capture $P2$

4.0.4 Detail of History Heuristic

The weight of history heuristic as follow

procedure update_cut_move(move, depth)
  history[move] += $2 \times depth$
  if history[move] $\geq bound_{history}$ **then**
    decrease()
procedure update_optimal_move(move, depth)
  history[move] += $depth^2$
  if history[move] $\geq bound_{history}$ **then**
    decrease()
procedure decrease
  for move $\in$ any moves in the board **do**
    history[move]= $\frac{\text{history[move]}}{2}$